

LedgerLang Compiler

A Financial Domain Specific Language & its Compiler Pipeline

Course	Compiler Design
Project	Option 2 — Self-designed DSL/Programming Language
Language	LedgerLang v1.0
Target	Python (executable output)
Date	April 2026

1. Abstract

This report documents the design and implementation of **LedgerLang**, a Domain Specific Language (DSL) tailored for financial computations and data analysis. The project implements a complete compiler pipeline comprising four stages: Lexical Analysis, Syntax Analysis (Parsing), Intermediate Code Generation, and Target Code Generation. LedgerLang allows users to declare financial variables such as accounts, rates, and portfolios; perform arithmetic and logical computations; and control program flow using conditional and iterative constructs. The compiler is implemented entirely in Python and produces both Three-Address Code (TAC) as an intermediate representation and executable Python code as the final target. The compiler also generates a numbered listing file interleaved with error messages, and continues analysis even in the presence of errors through robust error recovery mechanisms.

2. Introduction

A compiler is a program that translates source code written in a high-level language into a lower-level representation that can be executed by a machine. The study of compiler construction encompasses several fundamental areas of computer science including formal language theory, automata, data structures, and code optimization.

For this project, Option 2 was chosen: the design of an original Domain Specific Language. The chosen domain is **financial computing** — a field where custom scripting languages are commonly used in practice (e.g., Excel formulas, Bloomberg scripting, risk modelling tools). The resulting language, **LedgerLang**, provides intuitive constructs for declaring financial variables, computing derived values, evaluating conditions, and simulating iterative processes such as interest compounding or loan repayment schedules.

The objectives of this project are:

- Design a clear, unambiguous grammar for LedgerLang.
- Implement a Lexical Analyzer that tokenizes LedgerLang source code.
- Implement a Recursive Descent Parser that builds an Abstract Syntax Tree.
- Implement an Intermediate Code Generator producing Three-Address Code (TAC).
- Implement a Code Generator that produces executable Python from the AST.
- Produce a listing file with line-numbered source and interleaved error messages.
- Implement error recovery so compilation continues past errors.

3. Description of the Implemented Compiler

3.1 LedgerLang Language Design

LedgerLang is a statically structured, imperative language with financial semantics. Programs are sequences of statements. The language supports integers, floating-point numbers, strings, arrays (portfolios), boolean expressions, and a set of control flow constructs.

3.1.1 Keywords

Keyword	Purpose
ACCOUNT	Declare a financial account variable
RATE	Declare a rate/percentage variable
VAR	Declare a general variable
COMPUTE	Compute and assign a derived value
PORTFOLIO	Declare an array of values
IF / THEN / ELSE / END	Conditional branching
REPEAT / TIMES / END	Fixed-count loop
SHOW	Print a value (standard output)
WARN	Print a warning message
REPORT	Print a report/status message
AND / OR / NOT	Logical operators
TRUE / FALSE	Boolean literals

3.1.2 Grammar (BNF)

The formal grammar of LedgerLang is defined as follows:

```
program      -> statement*

statement    -> var_decl
              | portfolio_decl
              | compute_stmt
              | assign_stmt
              | if_stmt
              | repeat_stmt
              | output_stmt

var_decl     -> ('ACCOUNT' | 'RATE' | 'VAR') IDENTIFIER '=' expression
portfolio_decl -> 'PORTFOLIO' IDENTIFIER '=' '[' expr_list ']'
```

```

compute_stmt    -> 'COMPUTE' IDENTIFIER '=' expression
assign_stmt     -> IDENTIFIER '=' expression
if_stmt         -> 'IF' expression 'THEN' body ['ELSE' body] 'END'
repeat_stmt     -> 'REPEAT' expression 'TIMES' body 'END'
output_stmt     -> ('SHOW' | 'WARN' | 'REPORT') expression
body            -> statement*

expression      -> or_expr
or_expr         -> and_expr ('OR' and_expr)*
and_expr        -> not_expr ('AND' not_expr)*
not_expr        -> 'NOT' not_expr | comparison
comparison      -> additive (('==' | '!=' | '<' | '>' | '<=' | '>=') additive)*
additive        -> multiplicative (('+' | '-') multiplicative)*
multiplicative  -> unary (('*' | '/') unary)*
unary           -> '-' unary | primary
primary         -> '(' expression ')'
                | INTEGER | FLOAT | STRING
                | 'TRUE' | 'FALSE'
                | IDENTIFIER '[' expression ']'
                | IDENTIFIER

expr_list       -> expression (',' expression)*

```

3.2 Stage 1 — Lexical Analyzer (lexer.py)

The Lexical Analyzer reads the raw source code character by character and groups characters into meaningful units called **tokens**. Each token has a type and a value. The lexer handles the following token categories:

- **Keywords:** ACCOUNT, RATE, VAR, COMPUTE, IF, THEN, ELSE, END, REPEAT, TIMES, SHOW, WARN, REPORT, PORTFOLIO, AND, OR, NOT, TRUE, FALSE
- **Identifiers:** sequences of letters, digits, and underscores starting with a letter
- **Numbers:** integers (e.g., 1000) and floats (e.g., 0.05)
- **Strings:** double-quoted character sequences (e.g., "High debt")
- **Operators:** +, -, *, /, ==, !=, <, >, <=, >=
- **Delimiters:** (,), [,], ,, =
- **Comments:** lines starting with # are ignored

Example token output for the line `ACCOUNT savings = 5000`:

```

Token(KEYWORD,      'ACCOUNT', line=3)
Token(IDENTIFIER,   'savings', line=3)
Token(ASSIGN,       '=',      line=3)
Token(INTEGER,      5000,     line=3)

```

3.3 Stage 2 — Parser (parser.py)

The Parser takes the token stream from the Lexer and constructs an **Abstract Syntax Tree (AST)**. A Recursive Descent parsing strategy is used, where each grammar rule corresponds

directly to a Python method. The parser implements:

- Full expression precedence (OR < AND < NOT < comparison < additive < multiplicative < unary < primary)
- Nested if/else and repeat blocks
- Array index access expressions
- **Error recovery**: when a parse error is encountered, the parser records the error with its line number and skips ahead to the next recognizable statement-starting keyword, allowing compilation to continue.

Example AST for IF loan > 10000 THEN WARN "High debt" END:

```
If(
    condition = BinOp(Id(loan) > Num(10000)),
    then_body = [Warn(Str("High debt"))],
    else_body = None
)
```

3.4 Stage 3 — Intermediate Code Generator (icg.py)

The Intermediate Code Generator traverses the AST and produces **Three-Address Code (TAC)**. TAC is a linear sequence of instructions where each instruction has at most three operands. Temporary variables (t1, t2, ...) are generated for sub-expressions, and labels (L1, L2, ...) are used for control flow.

TAC instruction types generated:

Instruction Type	Example
Assignment	x = 5000
Binary operation	t1 = savings * tax
Conditional jump	IF_FALSE t3 GOTO L1
Unconditional jump	GOTO L2
Label definition	L1:
Array write	prices[0] = 100
Array read	t1 = prices[0]
Output	SHOW savings / WARN "msg"

3.5 Stage 4 — Code Generator (codegen.py)

The Code Generator traverses the AST a second time and produces executable **Python code** as the compilation target. Generating directly from the AST (rather than from TAC) allows the use of Python's structured control flow, resulting in clean and readable output code. The

generated code uses three helper functions — `ledger_show()`, `ledger_warn()`, and `ledger_report()` — to produce labelled output.

3.6 Listing File and Error Reporting

The compiler always produces a **listing file** — a numbered version of the source code with error messages interleaved immediately after the offending line. Even when errors are present, the compiler continues to analyze subsequent code. Target code is generated only when the source is error-free.

3.7 Project File Structure

```
ledgerlang/
■■■ compiler.py          # Main driver – orchestrates all 4 stages
■■■ lexer.py            # Stage 1: Lexical Analyzer
■■■ parser.py           # Stage 2: Recursive Descent Parser
■■■ ast_nodes.py        # AST node class definitions
■■■ icg.py              # Stage 3: Intermediate Code Generator (TAC)
■■■ codegen.py          # Stage 4: Python Code Generator
■■■ sample1.ledger      # Example: Personal finance calculator
■■■ sample2.ledger      # Example: Portfolio analysis
■■■ sample3_errors.ledger # Example: Error recovery demonstration
■■■ README.md           # Usage instructions
```

5. Results

This section presents the input source programs and the corresponding output at each stage of the compiler pipeline.

5.1 Sample Program 1 — Personal Finance Calculator

Input (sample1.ledger):

```
# LedgerLang Sample Program 1 - Personal Finance Calculator

ACCOUNT savings = 5000
ACCOUNT loan    = 20000
RATE  tax       = 0.15
RATE  interest  = 0.04

COMPUTE net = savings - (savings * tax)

IF loan > 10000 THEN
    WARN "High debt detected"
ELSE
    REPORT "Debt under control"
END

SHOW savings
SHOW net
SHOW loan

REPEAT 12 TIMES
    savings = savings + (savings * interest)
END

REPORT "Savings after 1 year:"
SHOW savings
```

Stage 1 Output — Tokens (selected):

Token(KEYWORD, 'ACCOUNT', line=3)	Token(IDENTIFIER, 'savings', line=3)
Token(ASSIGN, '=', line=3)	Token(INTEGER, 5000, line=3)
Token(KEYWORD, 'RATE', line=5)	Token(IDENTIFIER, 'tax', line=5)
Token(ASSIGN, '=', line=5)	Token(FLOAT, 0.15, line=5)
Token(KEYWORD, 'COMPUTE', line=9)	Token(IDENTIFIER, 'net', line=9)
Token(KEYWORD, 'IF', line=12)	Token(IDENTIFIER, 'loan', line=12)
Token(OP, '>', line=12)	Token(INTEGER, 10000, line=12)
Token(KEYWORD, 'THEN', line=12)	...

88 tokens produced, no errors.

Stage 2 Output — AST (selected nodes):

```
Program(
  VarDecl(ACCOUNT savings = Num(5000))
  VarDecl(ACCOUNT loan    = Num(20000))
```

```

VarDecl(RATE    tax      = Num(0.15))
VarDecl(RATE    interest= Num(0.04))
Compute(net = BinOp(Id(savings) - BinOp(Id(savings) * Id(tax))))
If(
    condition = BinOp(Id(loan) > Num(10000))
    then = [Warn(Str("High debt detected"))]
    else = [Report(Str("Debt under control"))]
)
Show(Id(savings)) Show(Id(net)) Show(Id(loan))
Repeat(count=Num(12), body=[
    Assign(savings = BinOp(Id(savings) + BinOp(Id(savings) * Id(interest))))
])
Report(Str("Savings after 1 year:"))
Show(Id(savings))
)
AST: 12 top-level statements, no errors.

```

Stage 3 Output — Three-Address Code (TAC):

```

1:  savings = 5000          2:  loan = 20000
3:  tax = 0.15             4:  interest = 0.04
5:  t1 = savings * tax      6:  t2 = savings - t1
7:  net = t2               8:  t3 = loan > 10000
9:  IF_FALSE t3 GOTO L1    10:  WARN "High debt detected"
11: GOTO L2               12:  L1:
13: REPORT "Debt under.."  14:  L2:
15: SHOW savings          16:  SHOW net
17: SHOW loan             18:  t4 = 0
19: L3:                   20:  t5 = t4 < 12
21: IF_FALSE t5 GOTO L4    22:  t6 = savings * interest
23: t7 = savings + t6      24:  savings = t7
25: t4 = t4 + 1           26:  GOTO L3
27: L4:                   28:  REPORT "Savings after 1 year:"
29: SHOW savings
29 TAC instructions generated, no errors.

```

Stage 4 Output — Generated Python Code:

```

savings = 5000
loan = 20000
tax = 0.15
interest = 0.04
net = (savings - (savings * tax))
if (loan > 10000):
    ledger_warn("High debt detected")
else:
    ledger_report("Debt under control")
ledger_show(savings)
ledger_show(net)
ledger_show(loan)
for _i in range(12):
    savings = (savings + (savings * interest))
ledger_report("Savings after 1 year:")

```



```
ledger_show(savings)
```

Execution Output:

```
[WARN]    High debt detected
[SHOW]    5000
[SHOW]    4250.0
[SHOW]    20000
[REPORT]  Savings after 1 year:
[SHOW]    8005.161092838402
```

5.2 Sample Program 2 — Portfolio Analysis

Input (sample2.ledger):

```
# LedgerLang Sample Program 2 - Portfolio Analysis

PORTFOLIO prices = [100, 250, 80, 320, 150]

VAR    budget      = 10000
RATE   commission  = 0.02
VAR    shares      = 40

COMPUTE cost  = shares * prices[0]
COMPUTE fee   = cost * commission
COMPUTE total = cost + fee

IF total > budget THEN
    WARN "Over budget!"
ELSE
    REPORT "Investment is within budget"
    SHOW total
END

VAR sell_price = 130
COMPUTE profit  = (sell_price - prices[0]) * shares
COMPUTE net_profit = profit - fee

SHOW net_profit

IF net_profit > 0 THEN
    REPORT "Profitable trade"
ELSE
    WARN "Loss on trade"
END
```

Execution Output:

```
[REPORT] Investment is within budget
[SHOW]   4080.0
[SHOW]   1120.0
[REPORT] Profitable trade
```

5.3 Sample Program 3 — Error Recovery Demonstration

Input (sample3_errors.ledger):

```
# Error demonstration — missing THEN keyword

ACCOUNT balance = 1000
IF balance > 500
    SHOW balance
END
```

```
ACCOUNT net = 900
SHOW net
```

Listing File Output (with errors interleaved):

```
1 | # Error demonstration – missing THEN keyword
2 |
3 | ACCOUNT balance = 1000
4 | IF balance > 500
   >>> ERROR: Line 5: Expected 'THEN' but got 'SHOW'
   >>> ERROR: Line 4: Unexpected token 'END'
5 |     SHOW balance
6 | END
7 | ACCOUNT net = 900
8 | SHOW net
```

Compiler behaviour:

- Error on line 4 is detected and reported with exact line number.
- Parser recovers and continues — lines 7 and 8 are still parsed correctly.
- Code generation is skipped because errors are present.
- 3 statements were successfully recovered after the error.

6. Discussion

6.1 Novelties

LedgerLang introduces several notable design choices that distinguish it from a generic teaching-language compiler:

- **Finance-semantic keywords:** Using ACCOUNT, RATE, PORTFOLIO, COMPUTE, WARN, and REPORT makes programs self-documenting and highly readable in a financial context — a domain expert can read a LedgerLang program without compiler knowledge.
- **Dual output pipeline:** The compiler generates both TAC (for academic inspection of the intermediate representation) and clean, structured Python code (as the executable target), making it useful both for teaching and for actual computation.
- **Error recovery with listing file:** Unlike many student compilers that stop at the first error, LedgerLang's parser skips to the next statement-starting keyword, allowing the full program to be analyzed and reported in a single pass.
- **Array support:** The PORTFOLIO construct and index expressions (prices[0]) provide array capabilities commonly needed in financial modelling.
- **Clean separation of concerns:** Each compiler stage is implemented in a distinct module (lexer.py, parser.py, icg.py, codegen.py), making the codebase readable and extensible.

6.2 Limitations

- **No type system:** LedgerLang currently does not perform type checking. Mixing strings with arithmetic expressions will result in a Python runtime error rather than a compile-time error.
- **No functions or procedures:** The language does not support user-defined functions. This limits code reuse for larger programs.
- **Single-file programs only:** There is no import/module system. All code must reside in a single .ledger file.
- **Integer and float only:** No support for complex number types, currency formatting, or date arithmetic — features that would be valuable in a real financial DSL.
- **No semantic analysis pass:** Variables can be used before being declared, and undefined variable references will only be caught at runtime.
- **Basic error recovery:** The recovery strategy (skip to next statement keyword) may sometimes skip too much code, causing cascade errors to be missed.

7. Conclusion

This project successfully demonstrates the full implementation of a compiler pipeline for **LedgerLang**, an original Domain Specific Language designed for financial computations. All four required compiler stages — Lexical Analysis, Syntax Analysis, Intermediate Code Generation, and Target Code Generation — have been implemented and tested.

The compiler correctly processes valid LedgerLang programs through all stages, producing Three-Address Code as an intermediate representation and executable Python code as the final output. It also handles erroneous inputs gracefully, reporting errors with line numbers and recovering to continue analysis of the remaining source code.

The choice of a finance-domain DSL proved well-motivated: the resulting language is both technically complete enough to satisfy all compiler design requirements and practically meaningful enough to demonstrate to a non-technical audience. The project deepened understanding of formal grammars, recursive descent parsing, abstract syntax trees, and the transformation pipeline that sits between human-readable source code and executable output.

Future work could extend LedgerLang with a semantic analysis phase, user-defined functions, a type system, and a web-based IDE — each of which would further strengthen both the language and the compiler as a teaching and practical tool.